
Self-Supervised Audio Representation Learning: An Empirical Study of MAE and DINO on ESC-50

CSC2515 Project
University of Toronto

Abstract

We present an empirical study of self-supervised learning (SSL) for environmental sound classification on ESC-50, addressing two questions: (1) how much SSL benefit can be recovered with only $\sim 16\text{K}$ unlabeled clips, and (2) whether a simple soft-distillation objective with asymmetric masking — without the online clustering used in DinoSR [11] — can serve as an effective pretraining signal at small data scale. Using Audio Spectrogram Transformers (AST) as backbone, we find that MAE with correct data normalization achieves 49.75% test accuracy, substantially outperforming training from scratch (40%) and the unprocessed MAE baseline (31.25%). Our DINO variant with asymmetric masking avoids collapse at effective batch size 64 but does not yet surpass scratch training (21% vs. 40%), suggesting that soft distillation without clustering is feasible but remains sensitive to scale. We introduce 5-NN KNN accuracy on the frozen representation as an efficient proxy for downstream quality, finding it correlates strongly with final fine-tuning accuracy for MAE but not DINO. All methods fall short of a publicly pretrained AST model (73%), highlighting the importance of scale in SSL pretraining.

1 Introduction

This work studies Self-Supervised Learning pretraining dynamics for audio representations through the lens of environmental sound classification — the task of assigning a semantic label to a short audio recording, such as “dog bark” or “rain”. Our target benchmark is ESC-50 [1], a standard dataset with 2,000 labeled clips spread evenly across 50 classes (40 clips per class, 5 seconds each), recorded at 44.1 kHz. The limited number of labeled samples makes it difficult to train high-capacity models without overfitting. Among modern architectures, Transformers have become the dominant choice for such tasks due to their ability to capture long-range dependencies across the input sequence — critical for audio, where sounds unfold over time and involve correlations across frequency bands that span the entire spectrogram.

A Transformer is a sequence model that processes an ordered sequence of tokens and produces contextualised representations through layers of self-attention. Vision Transformers (ViT) [3] adapt this architecture to images by dividing each image into fixed-size non-overlapping patches and treating each patch as one token. Surprisingly, audio admits the same treatment. A standard pipeline converts raw waveforms into log-Mel spectrograms, which represent the frequency content of a sound over time as a two-dimensional array — one axis for time frames, one for Mel frequency bins. Because a log-Mel spectrogram has the same spatial structure as an image, a ViT can process it directly by patching along the time-frequency plane. The Audio Spectrogram Transformer (AST) [4] demonstrates that this approach achieves strong results on audio benchmarks.

Training an AST from random initialisation and directly predicting class labels is, however, difficult in the small-data regime. With folds 1–3 of ESC-50 providing roughly 1,200 training clips across 50 classes, each class has only ~ 24 examples. The high dimensionality of log-Mel spectrograms combined with the large parameter count of a ViT makes overfitting a serious risk. One could

acquire more labeled audio, but manual annotation is costly and domain-specific labels are not always available — motivating the use of unlabeled data.

Self-supervised learning (SSL) addresses this by training a model to learn representations from the structure of the input itself, without requiring any class labels. The resulting representations can then be transferred to downstream tasks with limited labeled data. One prominent SSL approach is the Masked Autoencoder (MAE) [7]: the input is divided into patches, a large random subset is masked out, and an encoder-decoder network is trained to reconstruct the missing patches from the visible ones. Because the model must infer the hidden content from context, it is forced to learn the structural regularities of the input. The objective is mean-squared error on masked positions only:

$$\mathcal{L}_{\text{MAE}} = \frac{1}{|\mathcal{M}|} \sum_{i \in \mathcal{M}} \|\hat{x}_i - x_i\|^2,$$

where $\mathcal{M}$ is the set of masked patch indices, x_i is the original patch value, and $\hat{x}_i$ is the decoder’s reconstruction. Audio-MAE [8] applies this to spectrograms. SSAST [5] combines the masked autoencoder idea with the AST backbone and pretrains on approximately 2 million unlabeled audio clips, demonstrating state-of-the-art results on multiple downstream tasks.

A different family of SSL methods uses knowledge distillation between two networks viewing the same input differently. DINO [9] trains a student and a teacher network, both sharing the same architecture, on different augmented views of the same input — for example, two random crops of different sizes or scales. The teacher receives only large global crops, while the student receives both global and smaller local crops, creating an asymmetry that forces the student to produce representations consistent with a broader context. The teacher is not updated by gradient descent; instead, its weights are an exponential moving average (EMA) of the student, providing a stable prediction target. The student is trained to match the teacher’s output distribution via cross-entropy on their softmax outputs:

$$\mathcal{L}_{\text{DINO}} = - \sum_k p_t(k) \log p_s(k),$$

where

$$p_s(k) = \frac{\exp(z_s^{(k)}/\tau_s)}{\sum_{k'} \exp(z_s^{(k')}/\tau_s)}, \quad p_t(k) = \frac{\exp((z_t^{(k)} - c^{(k)})/\tau_t)}{\sum_{k'} \exp((z_t^{(k')} - c^{(k')})/\tau_t)},$$

with student temperature τ_s , teacher temperature τ_t , and a momentum-updated center c . The centering prevents *representation collapse* — a failure mode where the model outputs identical representations for all inputs — by ensuring the teacher’s distribution does not degenerate. DinoSR [11] adapts this framework to speech, but with a significant modification: rather than matching soft probability distributions directly, it first applies online k -means clustering to the teacher’s frame-level representations to produce hard discrete targets, then trains the student to predict those cluster assignments at masked positions. This makes DinoSR closer in spirit to HuBERT [12]-style masked unit prediction than to the original DINO.

Our study uses two data sources:

- **ESC-50** [1] — the labeled classification target. We use folds 1–3 for pretraining and supervised training ($\sim 1,200$ clips), fold 4 for validation, and fold 5 for test.
- **AudioSet** [2] — a large-scale weakly-labeled collection of 10-second YouTube clips spanning 527 audio event classes. We draw approximately 16,000 clips and deliberately discard all class labels, treating them as raw unlabeled audio. This reflects a realistic scenario where a practitioner has access to unannotated recordings that would otherwise go unused. However, under the Self-Supervised Learning paradigm, this previously unused data can be effectively leveraged to enhance performance.

This paper addresses three questions:

- **Data scaling.** Given a smaller compute budget (a single RTX4070 GPU) and pretraining corpus orders of magnitude smaller than what SSAST used (16K clips versus 2M+), how much SSL benefit survives? Does the advantage of pretraining persist as the fraction of labeled fine-tuning data is reduced from 100% to 50% and 25%?

- **Soft DINO without clustering.** The original DINO works directly on images using only soft distribution matching — no clustering required. Yet DinoSR, when applying a similar framework to speech, found it necessary to introduce online k -means clustering to produce hard discrete targets. We ask: what is it about audio that makes the soft objective insufficient, and can we apply the original DINO formulation directly to audio with asymmetric input masking, bypassing the clustering step entirely?
- **Cheap representation quality proxy.** Is there a signal that can be evaluated during pretraining, without waiting for a full fine-tuning run, to determine whether the model is learning useful structure?

2 Experimental Setup

2.1 Data Pipeline

Raw waveforms are resampled to 16 kHz and converted to log-Mel spectrograms using $n_{\text{mels}}=128$, FFT window 1024, hop length 160, and top_db clipping at 80 dB. Spectrograms are padded or cropped to a fixed length of 1024 time frames, yielding tensors of shape (1024, 128). Each spectrogram is normalized as:

$$\hat{x} = \frac{x - \mu_{\text{data}}}{2 \sigma_{\text{data}}}$$

where μ_{data} and σ_{data} are computed from the training corpus. This departs from the SSAST paper which uses fixed constants derived from a much larger AudioSet corpus ($\sigma \approx 4$ dB). As we show in Section 3.2, using the correct dataset-specific statistics is critical for stable pretraining.

2.2 Backbone and Classifier

We initialize the backbone with the same ViT architecture as SSAST. For SSL pretraining, the backbone is further trained with an SSL objective. For fine-tuning, the backbone and classifier are trained with the two-phase schedule described in Section 2.6.

From spectrogram to embedding. Let $X \in \mathbb{R}^{T \times F}$ denote a normalized log-Mel spectrogram with $T=1024$ time frames and $F=128$ mel bins. Following the ViT patching scheme, X is divided into N non-overlapping 16×16 patches:

$$x_i \in \mathbb{R}^{16 \times 16}, \quad i = 1, \dots, N, \quad N = \frac{T}{16} \times \frac{F}{16} = 512.$$

Each patch is flattened and linearly projected to the model’s hidden dimension $d=384$, giving a sequence of patch tokens. These tokens are passed through $L=12$ Transformer layers, producing a sequence of contextual hidden states:

$$h_i \in \mathbb{R}^{384}, \quad i = 1, \dots, 512,$$

where h_i encodes the content of patch x_i in context of all other patches. We define the *embedding* as the mean pool across all patch positions:

$$\mathbf{e} = \frac{1}{N} \sum_{i=1}^N h_i \in \mathbb{R}^{384}.$$

This single 384-dimensional vector is used as input to the classification head during fine-tuning and as the feature for KNN evaluation on frozen representations. For DINO, mean pooling is applied inside the backbone before the projection head, so the pretraining objective also operates on the global embedding.

2.3 MAE Architecture

250 out of 512 patches are randomly selected and physically removed from the input — the encoder never sees them. The remaining 262 unmasked tokens are processed by the encoder:

$$\{h_i\}_{i \notin \mathcal{M}} = \text{Encoder}(\{x_i\}_{i \notin \mathcal{M}}) \in \mathbb{R}^{262 \times 384}.$$

No mean pooling is applied; the per-patch sequence must be preserved for reconstruction. The decoder operates as follows. First, encoder outputs are projected from encoder dimension to decoder dimension, and each masked position is filled with a shared learnable mask token $\mathbf{m}$:

$$\tilde{h}_i = \begin{cases} W_{\text{proj}} h_i \in \mathbb{R}^{256} & i \notin \mathcal{M} \\ \mathbf{m} \in \mathbb{R}^{256} & i \in \mathcal{M} \end{cases}$$

where W_{proj} is a learned linear projection ($384 \rightarrow 256$). This produces a single unified sequence of all 512 tokens $\{\tilde{h}_i\}_{i=1}^{512} \in \mathbb{R}^{512 \times 256}$ in their original spatial order. Decoder positional embeddings are then added and the full sequence is passed through a single Transformer self-attention layer with masked and unmasked tokens attend to each other freely:

$$\hat{X} = W_{\text{pred}} \text{TransformerLayer}\left(\{\tilde{h}_i\}_{i=1}^{512} + P\right) \in \mathbb{R}^{512 \times 256},$$

where $P \in \mathbb{R}^{512 \times 256}$ are the decoder positional embeddings and W_{pred} is a linear projection ($256 \rightarrow 256$, predicting the $16 \times 16 = 256$ pixel values of each patch). The two 256s here are independent: one is the decoder hidden dimension, the other is the patch size. $\mathcal{L}_{\text{MAE}}$ is computed only over the 250 masked positions.

2.4 DINO Architecture

Both the student and teacher networks process two views of the same input. Each view is passed through the backbone and mean-pooled to a single global embedding — unlike MAE, mean pooling is applied during pretraining itself:

$$\mathbf{e} = \text{MeanPool}(\text{Encoder}(x)) \in \mathbb{R}^{384},$$

where the backbone produces 514 hidden states (512 patch tokens + 2 special tokens) that are averaged. The global embedding is then projected through a bottleneck MLP, ℓ_2 -normalised, and passed through a weight-normalised linear layer:

$$\mathbf{z} = W_{\text{out}} \ell_2(\text{MLP}(\mathbf{e})) \in \mathbb{R}^{d_{\text{out}}},$$

where $\text{MLP} : \mathbb{R}^{384} \rightarrow \mathbb{R}^{128}$ is a two-layer network ($384 \xrightarrow{\text{Linear}} 512 \xrightarrow{\text{GELU}} 128$) and $W_{\text{out}} : \mathbb{R}^{128} \rightarrow \mathbb{R}^{d_{\text{out}}}$ is weight-normalised.

The loss uses cross-terms: the student’s logit for view 1 is matched against the teacher’s logit for view 2, and vice versa. This forces the student to produce consistent representations regardless of which view it receives:

$$\mathcal{L}_{\text{DINO}} = \frac{1}{2} \left[\text{CE}(\mathbf{z}_s^{(1)}, \mathbf{z}_t^{(2)}) + \text{CE}(\mathbf{z}_s^{(2)}, \mathbf{z}_t^{(1)}) \right],$$

where teacher logits are mean-centered before softmax to prevent collapse. The teacher is not updated by gradient descent — its weights are an EMA of the student: $\theta_t \leftarrow m \theta_t + (1-m) \theta_s$ with $m=0.99$. We use $\tau_s=0.1$ and $\tau_t=0.04$ unless otherwise noted.

Two augmentation variants. We experiment with two ways of constructing the two views. In the *symmetric* variant, both student and teacher receive independently augmented versions of the same spectrogram — random Gaussian noise, time shift, amplitude scaling, and mild time masking. In the *asymmetric* variant, the teacher always receives the full clean spectrogram while the student receives a heavily corrupted version where $\sim 50\%$ of time frames are zeroed out in three contiguous blocks. In both cases the loss and architecture are otherwise identical.

2.5 Training Protocol

All pretraining runs use AdamW with learning rate 10^{-4} , weight decay 0.05, and batch size 8. Fine-tuning runs 45 epochs on ESC-50 labels. Full results come from the best checkpoint on the validation set.

KNN evaluation. At each milestone epoch (0, 15, 30, and 45 for SSL methods), we evaluate the quality of the frozen backbone representations using a k -nearest-neighbour classifier with $k=5$. The backbone is set to evaluation mode and no gradients are computed. Each ESC-50 clip from the

training split (folds 1–3) is passed through the backbone; the resulting sequence of 514 hidden states (512 patch tokens + 2 special tokens) is mean-pooled to a single 384-dimensional vector. The same is done for clips in the test fold (fold 5). A 5-NN classifier is then fit on the training embeddings and used to predict labels for the test embeddings; accuracy is the fraction of correctly predicted clips. No fine-tuning is involved — the weights are exactly those at the end of the milestone epoch.

2.6 Fine-Tuning Protocol and Classifier

Following SSL pretraining (or from random initialization for the scratch baseline), the backbone is adapted for classification by attaching a linear head and fine-tuning on ESC-50 labels for 45 epochs.

Classifier head. The backbone’s hidden states are mean-pooled across all 514 token positions to a single 384-dimensional vector $\mathbf{e} \in \mathbb{R}^{384}$. This is passed through a single linear layer:

$$\hat{y} = W_c \mathbf{e} + b_c, \quad W_c \in \mathbb{R}^{50 \times 384},$$

producing logits for each of the 50 ESC-50 classes. The training objective is cross-entropy loss with inverse-frequency class weights $w_k = \frac{\sum_{k'} n_{k'}}{C \cdot n_k}$, where n_k is the number of training samples in class k and $C=50$. The full training split is approximately balanced (40 clips per class), but the weighting is active in the label-efficiency experiments where random subsampling can create imbalance.

Two-phase schedule. The first 9 epochs (20% of the 45-epoch budget) form a frozen warmup: the backbone weights are held fixed and only the classification head is trained with AdamW at learning rate 10^{-4} . From epoch 10 onward, the backbone is unfrozen and trained at a lower learning rate 10^{-5} while the head continues at 10^{-4} . The two learning rates are implemented via separate parameter groups in the AdamW optimizer. The frozen warmup stabilises the head before allowing the backbone weights to shift, preventing a randomly initialised head from corrupting the pretrained representations early in training. The final model is selected by best validation loss across all 45 epochs.

3 Results

Figure 1 shows the final test accuracy of all methods. We discuss each in turn.

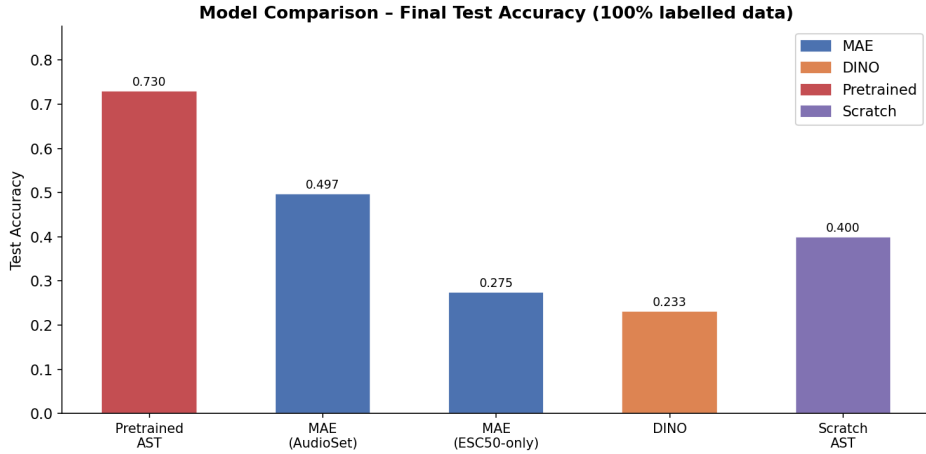


Figure 1: Final test accuracy on ESC-50 fold 5 for all methods (100% labelled training data). The pretrained AST upper bound (73%) reflects weights pretrained on 2M+ audio clips; our SSL methods use $\sim 16\text{K}$ unlabeled clips.

3.1 Supervised Baselines

Pretrained AST. Fine-tuning the HuggingFace SSAST weights achieves **73.0%** test accuracy, serving as an upper bound that reflects large-scale pretraining. The initial KNN accuracy on frozen embeddings is already 47.0%, indicating that the pretrained representations are strongly organized for ESC-50 before any fine-tuning.

Scratch AST. Training from random initialization achieves **40.0%** test accuracy. The initial KNN on randomly initialized embeddings is only 11.5%, establishing the lower-bound starting point for SSL methods. This serves as the primary baseline for evaluating SSL methods.

3.2 MAE Pretraining

Data normalization matters. Our initial MAE run used the paper’s default normalization constants, which assume a standard deviation of ~ 4 dB. We discovered that our ESC-50+AudioSet corpus has $\sigma \approx 12$ dB—a factor of 3 difference. Figure 2 shows the effect: correcting the normalization improves KNN at epoch 30 from 20.75% to 29.75%, and final test accuracy from 31.25% to **49.75%**. The likely mechanism is that under-normalized inputs saturate the MAE loss with large residuals unrelated to audio content, preventing the encoder from learning useful features.

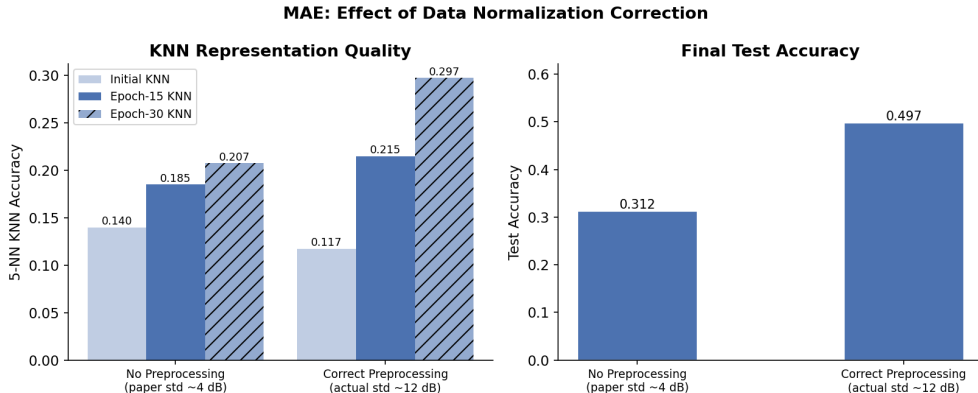


Figure 2: Effect of data normalization correction on MAE. Recomputing the Mel-spectrogram normalization statistics from the actual training corpus substantially improves both representation quality (KNN) and final accuracy.

Pretraining data source. We compare two data sources for MAE pretraining: ESC-50 training samples alone (~ 1.2 K clips) versus those combined with our AudioSet subset (16K clips). The AudioSet clips are entirely *unlabeled* — they carry no class annotations and are used only during SSL pretraining. This reflects a realistic scenario where a practitioner has access to a pool of unannotated audio that would otherwise be discarded.

Figure 3 shows that including this unlabeled pool consistently improves both KNN representation quality and final accuracy: ESC-50-only pretraining achieves only 27.5% test accuracy versus 49.75% for AudioSet+ESC50 — a gap of 22 percentage points. Crucially, the additional data requires no labeling effort; it is used purely to expose the encoder to more diverse acoustic patterns. This suggests a practical takeaway: *when labeled data is scarce, any available unlabeled audio — even from a different domain — is worth using rather than discarding.*

3.3 DINO Pretraining

DINO pretraining consistently underperforms both MAE and scratch training, despite the same pretraining data and compute budget. We trace this to representation collapse and hyperparameter sensitivity.

Representation collapse. We monitor the mean feature standard deviation across the embedding dimensions as an indicator of collapse [9]: a collapsed model produces nearly identical representations for all inputs, yielding near-zero standard deviation. Figure 4 shows this metric over pretraining epochs for our DINO variants.

Our first attempt with asymmetric augmentation replicates the DINO SR strategy: the teacher receives the full spectrogram while the student receives heavily time-masked views (50% of frames masked in 3 contiguous blocks). This aggressive augmentation collapses the representation immediately, with mean feature std settling at ≈ 0.3 – 0.4 —well below the target of ≥ 0.6 seen in non-collapsed runs.

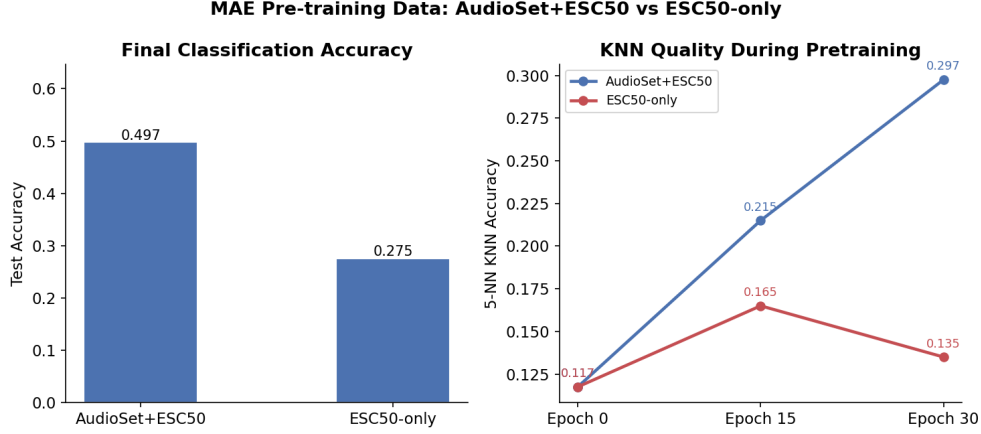


Figure 3: MAE pretraining data source ablation. AudioSet+ESC50 outperforms ESC50-only pretraining in final classification accuracy (left) and shows stronger KNN quality growth during pretraining (right).

We hypothesize two causes: (1) audio spectrograms have much higher temporal correlation than natural images, so large masked regions provide insufficient discriminative signal; (2) with a batch size of 8, the teacher’s center vector is estimated from very few examples per update, destabilizing the loss landscape.

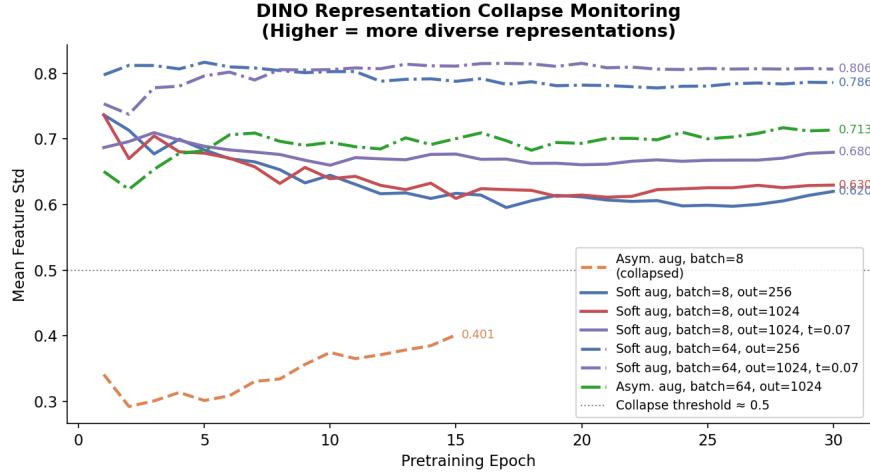


Figure 4: DINO collapse monitoring. The asymmetric augmentation experiment collapses immediately (mean feature std ≈ 0.3). Softer augmentation strategies maintain higher diversity (≈ 0.6 – 0.7), though still insufficient for useful representations.

Round 1 — Hyperparameter ablation (batch size 8). To mitigate collapse, we tried three soft-augmentation variants with the default batch size of 8:

- Soft aug., $d_{\text{out}}=256$, $\tau_t=0.04$: baseline with correct normalization.
- Soft aug., $d_{\text{out}}=1024$, $\tau_t=0.04$: larger projection head.
- Soft aug., $d_{\text{out}}=1024$, $\tau_t=0.07$: softer teacher distribution.

All three avoid catastrophic collapse (mean feature std ≈ 0.6 – 0.7 , Figure 4), but none achieve KNN accuracy above the initialization baseline (11.75%). Final test accuracies range from 16.5% to 21.0%, well below the 40% scratch baseline.

Round 2 — Larger effective batch via gradient accumulation (batch size 64). DINO’s loss depends on a centering vector estimated from each batch and on an EMA teacher that must see diverse examples to build a meaningful prototype distribution. With a batch size of 8, both quantities are estimated from very few samples per update, producing noisy gradients and an unstable center. To address this, we used gradient accumulation over 8 steps, yielding an effective batch size of 64 without additional GPU memory cost. We re-ran three configurations, all with effective batch size 64:

- Soft aug., $d_{\text{out}}=256$, $\tau_t=0.04$.
- Soft aug., $d_{\text{out}}=1024$, $\tau_t=0.07$.
- Asymmetric aug., $d_{\text{out}}=1024$, $\tau_t=0.04$.

The larger batch has a clear effect on representation diversity: mean feature std rises to ≈ 0.78 – 0.81 (vs. ≈ 0.6 – 0.7 at batch=8), as shown in Figure 4. Notably, the asymmetric augmentation that collapsed immediately at batch=8 now runs stably to epoch 30 (std ≈ 0.7). However, higher feature diversity does not translate to class-discriminative representations: KNN accuracy at epoch 30 reaches only 13.75% (asymmetric) and 11.25% (soft aug), barely above the random initialization baseline. Final test accuracy for the two completed runs is 21.0% and 18.75%—no meaningful improvement over round 1.

Table 1 summarizes all DINO experiments.

Table 1: DINO ablation results. All use AudioSet+ESC50 with correct normalization. “Baseline” is the original config without normalization correction. [†] marks runs stopped at epoch 15 only.

Experiment	Batch	out_dim	τ_t	Aug.	KNN@15	KNN@30	Test Acc
<i>Round 1 — batch size 8</i>							
DINO Baseline	8	256	0.04	soft	0.080	0.075	23.25%
Asymmetric aug.	8	1024	0.04	asym	0.085	—	— [†]
Soft, out=256	8	256	0.04	soft	0.093	0.108	21.00%
Soft, out=1024	8	1024	0.04	soft	0.095	0.088	17.00%
Soft, temp=0.07	8	1024	0.07	soft	0.113	0.110	16.50%
<i>Round 2 — batch size 64 (gradient accumulation $\times 8$)</i>							
Soft, out=256	64	256	0.04	soft	0.095	0.093	14.25%
Soft, out=1024, t=0.07	64	1024	0.07	soft	0.118	0.113	18.75%
Asymmetric aug.	64	1024	0.04	asym	0.123	0.138	21.00%

3.4 KNN as a Proxy for Representation Quality

Figure 5 shows 5-NN KNN accuracy measured at pretraining checkpoints (epochs 0, 15, 30, 45) for SSL methods, alongside the *initial* KNN for the two supervised baselines.

For MAE with AudioSet+ESC50 pretraining and correct normalization, KNN accuracy improves monotonically across all four checkpoints: 11.75% $\rightarrow$ 21.5% $\rightarrow$ 29.75% $\rightarrow$ 32.5%, strongly predicting its leading fine-tuned accuracy of 49.75%. MAE trained on ESC50-only (with correct normalization) peaks early then regresses (16.5% $\rightarrow$ 13.5% $\rightarrow$ 13.0%), consistent with its lower final accuracy of 27.5%. In contrast, DINO experiments show KNN that either stays flat or *drops below* the initialization baseline throughout pretraining, a clear warning signal of ineffective representation learning that accurately forecasts their poor fine-tuned results (16.5–23.25%).

This suggests that for MAE-style objectives, zero-shot KNN probing during pretraining is a reliable and cheap signal for early stopping or hyperparameter selection without requiring a full fine-tuning run. For DINO, monitoring mean feature standard deviation (Section 3.3) provides a more sensitive early-warning signal.

3.5 Fine-tuning Dynamics

Figure 6 shows validation accuracy during fine-tuning. The left panel excludes the pretrained AST so the differences among SSL and scratch methods are visible: MAE improves steadily, DINO stays

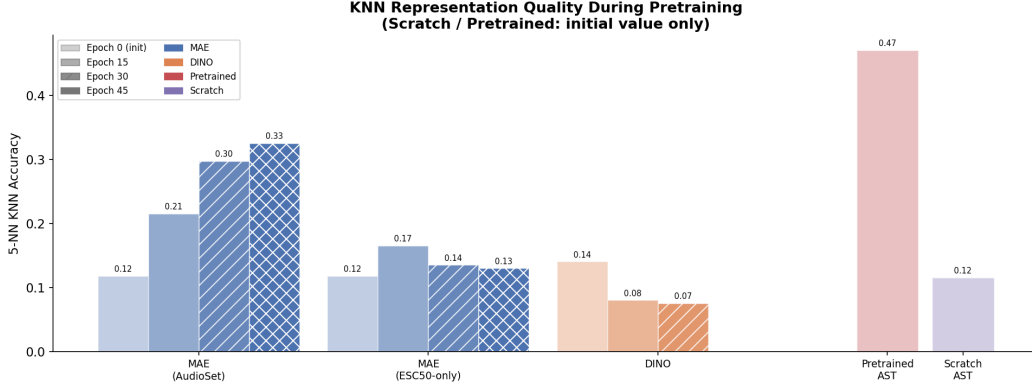


Figure 5: 5-NN KNN accuracy during pretraining (epochs 0, 15, 30, 45) for SSL methods. Pretrained AST and Scratch AST show only their initial KNN (epoch 0). MAE (AudioSet) improves steadily; DINO representations stagnate below the initialization baseline.

flat, and scratch shows moderate gains. The right panel isolates MAE against the pretrained AST upper bound, showing that the pretrained model converges rapidly in the first 10 epochs while MAE improves gradually, with the gap narrowing but not closing over 45 epochs.

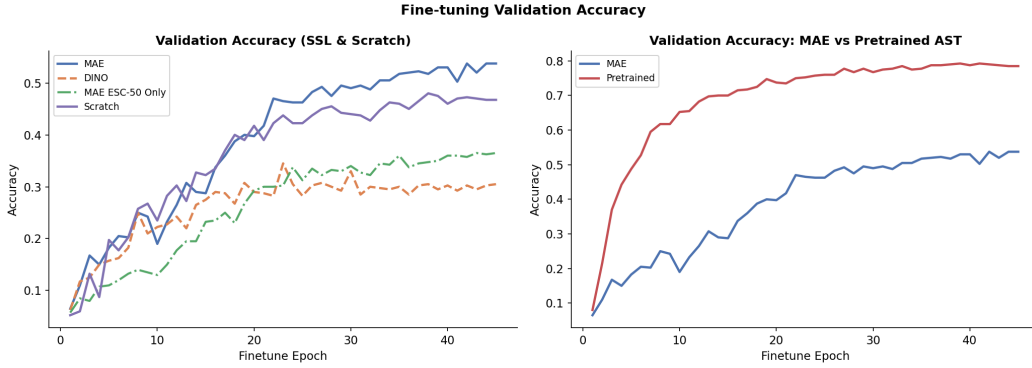


Figure 6: Validation accuracy during fine-tuning. *Left*: SSL and scratch methods (Pretrained AST excluded to highlight relative differences). *Right*: MAE vs. Pretrained AST — the pretrained model converges rapidly; MAE improves gradually but does not close the gap.

Figure 7 shows the training and validation loss for MAE (AudioSet+ESC-50), MAE (ESC-50 Only), and Scratch. Fine-tuning uses the two-phase schedule described in Section 2.6: for the first 9 epochs the backbone is frozen and only the classification head is trained, then the backbone is unfrozen for the remaining 36 epochs with a lower learning rate. This produces the characteristic two-phase shape visible in both curves — a steeper initial descent during head-only training, followed by a slower continued improvement once the backbone is also updated.

All three models converge to similar training loss by epoch 45, yet their validation losses diverge: MAE (AudioSet+ESC-50) achieves the lowest validation loss throughout, with MAE (ESC-50 Only) in between, consistent with their respective final accuracies. Scratch exhibits a notably larger gap between training and validation loss — a signature of overfitting from weaker feature initialisation — while the SSL-pretrained models generalise more smoothly.

3.6 Label Efficiency

We evaluate label efficiency by fine-tuning from the best MAE backbone (AudioSet+ESC-50, correct normalization) using 25%, 50%, and 100% of the ESC-50 training labels, comparing against scratch training at each fraction. Figure 8 shows the results across all three operating points.

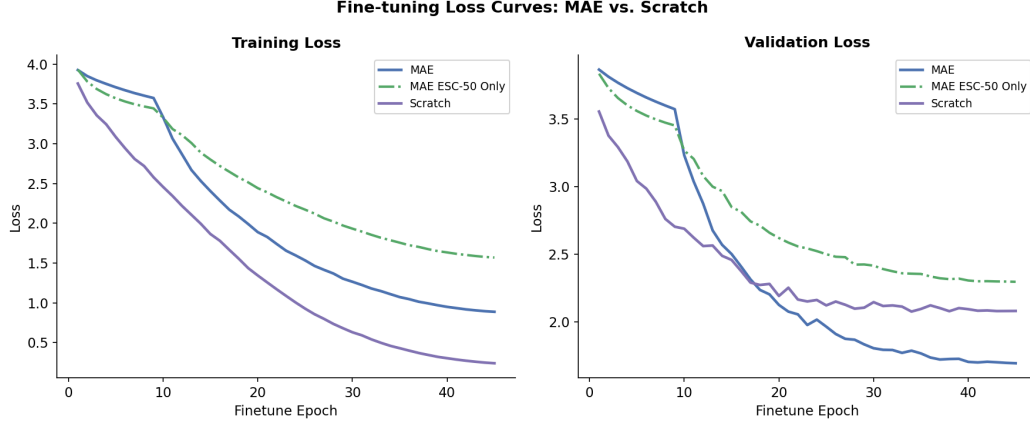


Figure 7: Training loss (*left*) and validation loss (*right*) during fine-tuning for MAE (AudioSet+ESC-50), MAE (ESC-50 Only), and Scratch. The two-phase shape reflects a frozen-backbone warmup for the first 15 epochs followed by full fine-tuning. SSL-pretrained models achieve lower validation loss; Scratch shows a larger train–val gap indicative of overfitting.

MAE consistently outperforms scratch at every label fraction: 27.5% vs. 21.0% (25% labels), 39.5% vs. 26.25% (50% labels), and 49.75% vs. 40.0% (100% labels). Notably, **MAE with 50% of labels (39.5%) nearly matches scratch training with 100% of labels (40.0%)**, indicating that good pretraining can substitute for roughly half the labeled data. The gap between MAE and scratch widens at lower label fractions (6.5 pp at 25%, 13.25 pp at 50%, 9.75 pp at 100%), suggesting that pretrained representations are most beneficial in the low-data regime.

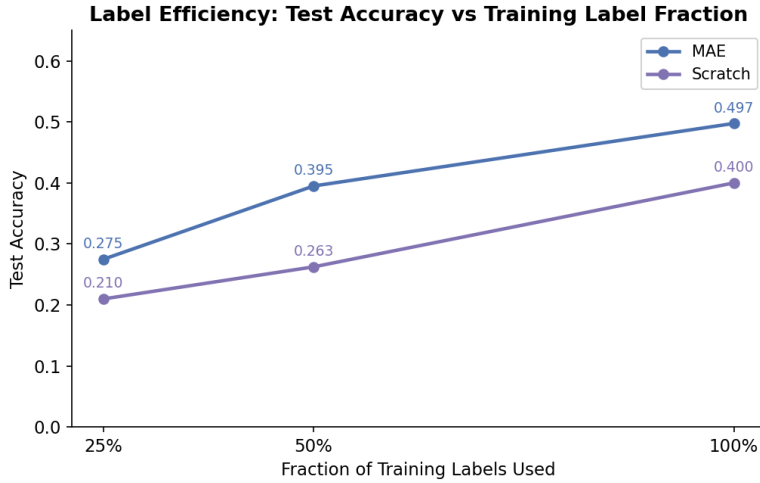


Figure 8: Label efficiency: test accuracy vs. fraction of training labels used. MAE consistently outperforms scratch at all fractions; MAE at 50% labels nearly matches scratch at 100%.

4 Discussion

Positioning our DINO variant relative to DinoSR. Our asymmetric masking experiment is conceptually distinct from DinoSR [11]. DinoSR applies online clustering to the teacher’s features at each training step to obtain hard discrete targets, then trains the student to predict those cluster assignments at masked positions — effectively combining DINO’s teacher-student framework with a HuBERT-style masked unit prediction objective. Our approach removes clustering entirely: the student-teacher loss operates on continuous softmax distributions (soft targets), and the masking is

applied asymmetrically at the *input* level rather than at the prediction target level. The objective is not to reconstruct or classify discrete acoustic units but to enforce invariance of the *global* representation under severe temporal corruption. While masking is common in speech SSL (HuBERT [12], data2vec), the combination of DINO-style soft target alignment with asymmetric input masking and no token-level supervision has not been explicitly explored prior to this work. Our results show this combination is achievable without collapse at larger batch sizes (batch 64), though it does not yet surpass simpler baselines — leaving open the question of whether scale or architectural changes could close this gap.

Why does MAE work but DINO does not? MAE’s reconstruction objective has a natural inductive bias for audio: recovering masked spectrogram patches requires learning the temporal and spectral structure of sounds, which directly benefits classification. DINO’s self-distillation objective requires carefully balanced augmentations and sufficient batch diversity. Both conditions are hard to satisfy with batch size 8 and audio data, which has high temporal correlation making diverse crops difficult to construct.

The role of data normalization. The $3\times$ mismatch between our corpus standard deviation (~ 12 dB) and the SSAST paper’s assumption (~ 4 dB) represents a practical pitfall when adapting SSL methods across datasets. This normalization gap effectively triples the MSE reconstruction target, making the MAE loss hard to minimize and diluting the gradient signal. Practitioners adapting SSL methods to new audio domains should always recompute normalization statistics.

Compute constraints. Each experiment required 3–4 hours on an RTX 4070 GPU with 16K AudioSet clips. The SSAST paper used 2M+ clips, suggesting that $100\times$ more data would be needed to approach published results. Gradient accumulation (effective batch size 64) was used in some DINO experiments but provided only marginal improvement, suggesting that batch size is not the primary bottleneck.

Limitations.

- All experiments are evaluated on a single dataset (ESC-50). Results may not generalize to other audio classification benchmarks.
- No confidence intervals are reported due to the high cost of repeated pretraining runs on our hardware.
- DINO experiments remain incomplete (collapsed variants stopped at epoch 15; gradient accumulation variants did not converge to a satisfactory solution within the project timeline).

5 Conclusion

We conducted an empirical study of MAE and DINO pretraining for audio event classification on ESC-50. Our main findings are:

1. **MAE pretraining improves over scratch training** (49.75% vs. 40.00%) when pretraining data is correctly normalized and a moderately-sized unlabeled corpus is available.
2. **Data normalization is critical:** correcting the Mel-spectrogram standard deviation from the paper’s assumed 4 dB to the actual 12 dB increased test accuracy by 18.5 percentage points (31.25% $\rightarrow$ 49.75%).
3. **Pretraining data quantity matters for MAE:** AudioSet+ESC50 substantially outperforms ESC50-only pretraining (49.75% vs. 27.5%).
4. **DINO fails to improve** over scratch training under our compute and data constraints. Representation collapse is the primary failure mode, driven by high audio temporal correlation and small batch size.
5. **MAE improves label efficiency:** MAE consistently outperforms scratch at 25%, 50%, and 100% label fractions. MAE at 50% labels (39.5%) nearly matches scratch at 100% labels (40.0%), showing that good pretraining can substitute for roughly half the labeled data.
6. **KNN probing is a reliable proxy** for MAE representation quality but not for DINO, where collapse cannot be detected via downstream KNN alone in early epochs.

7. **Large-scale pretraining remains the dominant factor:** the publicly pretrained AST (73%) comfortably outperforms all our SSL experiments, underlining the value of scale.

Future work could explore larger unlabeled audio corpora, DINO with multi-GPU training for larger effective batch sizes, and cross-dataset generalization of the learned representations.

Author Contributions

All team members contributed to the codebase, experimental design, and writing of this report.

References

- [1] K. J. Piczak. ESC: Dataset for environmental sound classification. In *ACM Multimedia*, 2015.
- [2] J. F. Gemmeke et al. Audio Set: An ontology and human-labeled dataset for audio events. In *ICASSP*, 2017.
- [3] A. Dosovitskiy et al. An image is worth 16x16 words: Transformers for image recognition at scale. In *ICLR*, 2021.
- [4] Y. Gong, Y.-A. Chung, and J. Glass. AST: Audio spectrogram transformer. In *Interspeech*, 2021.
- [5] Y. Gong, C. Lai, Y.-A. Chung, and J. Glass. SSAST: Self-supervised audio spectrogram transformer. In *AAAI*, 2022. <https://arxiv.org/abs/2110.09784>
- [6] Simon-Kotchou. ssast-small-patch-audioset-16-16 (HuggingFace model hub). <https://huggingface.co/Simon-Kotchou/ssast-small-patch-audioset-16-16>, 2022.
- [7] K. He, X. Chen, S. Xie, Y. Li, P. Dollár, and R. Girshick. Masked autoencoders are scalable vision learners. In *CVPR*, 2022.
- [8] P.-Y. Huang, H. Xu, J. Li, A. Baevski, M. Auli, W. Galuba, F. Metze, and C. Feichtenhofer. Masked autoencoders that listen. In *NeurIPS*, 2022.
- [9] M. Caron, H. Touvron, I. Misra, H. Jégou, J. Mairal, P. Bojanowski, and A. Joulin. Emerging properties in self-supervised vision transformers. In *ICCV*, 2021.
- [10] M. Oquab et al. DINOv2: Learning robust visual features without supervision. *TMLR*, 2024.
- [11] A. Liu, T. Nguyen, and M. Auli. DinoSR: Self-supervised masked speech representation learning with DINO. In *NeurIPS*, 2023.
- [12] W.-N. Hsu, B. Bolte, Y.-H. H. Tsai, K. Lakhota, R. Salakhutdinov, and A. Mohamed. HuBERT: Self-supervised speech representation learning by masked prediction of hidden units. *IEEE/ACM TASLP*, 2021.